

Research-Oriented University System

Discipline: Object-Oriented Programming and Design

Project type: Console-based Java application

Team members:

Kabdygali Aibyn

Alybaev Alikhan

Amir Bisultan

Cover

Research-Oriented University System

Project name: Research-Oriented University System

Discipline: Object-Oriented Programming and Design

University: KBTU

Team members:

- *Team member 1*
- *Team member 2*
- *Team member 3*
- *Team member 4*

Insert: KBTU logo or university image.

The Main Goal of the Project

The main goal of the project is to create a console-based university information system for a research-oriented university.

The system helps organize communication and work between students, teachers, managers, admins, employees, and researchers. It supports academic processes, research activities, reports, messages, logs, and data storage.

The project was created to apply the main concepts learned in Object-Oriented Programming and Design.

Project Description

The system is a console Java application. Each user enters the system through authentication and uses functions according to their role.

Main roles:

- Student;
- Teacher;
- Manager;
- Admin;
- Employee;
- Researcher.

Main functions:

- course registration;
- approval of registration requests;
- assigning teachers to courses;
- putting marks;
- viewing transcript;
- managing users;
- adding research papers;
- joining research projects;
- generating reports.

Diagrams

Before writing code, we created diagrams to design the system architecture.

We used:

1. Use Case diagram;
2. UML Class diagram.

The Use Case diagram shows what each actor can do in the system.

The Class diagram shows classes, fields, methods, inheritance, interfaces, and relationships between objects.

Insert: Use Case diagram screenshot.

Insert: UML Class diagram screenshot.

Use Case Diagram

Main actors:

- User;
- Student;
- Employee;
- Teacher;
- Manager;

- Admin;
- Researcher.

Main use cases:

- login and logout;
- view profile;
- register for courses;
- approve registration;
- assign courses to teachers;
- put marks;
- view transcript;
- manage users;
- add research papers;
- join research projects;
- view top-cited researcher.

Insert: final Use Case diagram.

Class Diagram

The main class hierarchy is:

```

User
├── Student
│   └── ResearchStudent
└── Employee
    ├── Teacher
    │   └── Professor
    ├── Manager
    ├── Admin
    └── ResearchEmployee
  
```

The most important design decision is that `Researcher` is an interface. This is because a student, teacher, or employee can be a researcher.

Insert: final UML Class diagram.

Working with Java Project

The project is divided into packages. This helps keep the code clean and organized.

```

src/
├── users/
└── academic/
  
```

```
|— research/  
|— services/  
|— storage/  
|— exceptions/  
|— enums/  
|— comparators/  
|— communication/  
|— reports/  
|— logs/  
|— factories/  
|— facade/  
└─ Main.java
```

Insert: screenshot of project structure from IDE.

Packages

We divided classes into several main packages:

- `users` — users and role hierarchy;
- `academic` — courses, lessons, marks, transcripts;
- `research` — research papers, projects, profiles;
- `services` — business logic;
- `storage` — data storage and serialization;
- `exceptions` — custom exceptions;
- `enums` — enumerations;
- `comparators` — sorting strategies;
- `communication` — messages and news;
- `logs` — user action logs.

This structure makes the project easier to understand and extend.

User Class

`User` is the base abstract class for all users.

```
public abstract class User implements Serializable, Comparable<User> {  
    private String id;  
    private String username;  
    private String password;  
    private String firstName;  
    private String lastName;  
    private String email;
```

```
        private UserRole role;
    }
```

Main methods:

```
checkPassword();
getFullName();
viewProfile();
compareTo();
equals();
hashCode();
toString();
```

`User` implements `Serializable` for saving data and `Comparable<User>` for sorting users.

Student Class

`Student` is a subclass of `User`.

```
public class Student extends User {
    private int yearOfStudy;
    private double gpa;
    private int totalCredits;
    private List<Course> enrolledCourses;
    private Transcript transcript;
    private Researcher researchSupervisor;
}
```

Student can:

- view courses;
- register for courses;
- view marks;
- view transcript;
- rate teachers.

A student cannot take more than 21 credits.

Registration Request

Course registration is not direct. It is done through a request.

Process:

Student selects course → Registration request is created → Manager approves/rejects → Student is enrolled

```
private Student student;  
private Course course;  
private RegistrationStatus status;  
private LocalDateTime createdAt;
```

`RegistrationStatus` values:

```
PENDING,  
APPROVED,  
REJECTED
```

Credit limit check:

```
if (student.getTotalCredits() + course.getCredits() > 21) {  
    throw new CreditLimitExceededException("Credit limit exceeded");  
}
```

Employee, Teacher, Manager, Admin

`Employee` is an abstract subclass of `User`.

```
public abstract class Employee extends User {  
    private double salary;  
    private LocalDate hireDate;  
}
```

`Teacher` can view courses, view students, and put marks.

`Manager` can approve registration, assign courses to teachers, manage news, and create reports.

`Admin` can add, remove, update users, and view user action logs.

This hierarchy avoids code duplication and keeps common employee data in one place.

Course and Lesson Classes

`Course` represents a university course.

```
public class Course implements Serializable {
    private String code;
    private String name;
    private int credits;
    private List<Teacher> instructors;
    private List<Student> students;
    private List<Lesson> lessons;
}
```

A course can have more than one instructor, so we use `List<Teacher>`.

`Lesson` represents one lesson.

```
private String topic;
private LocalDateTime dateTime;
private LessonType type;
```

`LessonType` values:

```
LECTURE,
PRACTICE
```

Mark and Transcript

`Mark` stores a student's result for one course.

```
private Course course;
private double firstAttestation;
private double secondAttestation;
private double finalExam;
```

Total mark:

```
public double getTotal() {
    return firstAttestation + secondAttestation + finalExam;
}
```

`Transcript` stores all marks of a student.

```
private Student student;  
private List<Mark> marks;
```

It can print transcript and calculate GPA.

Insert: screenshot of transcript output.

Researcher Design

`Researcher` is an interface, not a class.

Reason: researcher is a role. Different users can be researchers:

- student;
- teacher;
- employee;
- professor.

Java does not support multiple inheritance of classes, so interface is the best solution.

```
public interface Researcher {  
    ResearchProfile getResearchProfile();  
    void printPapers(Comparator<ResearchPaper> comparator);  
    int getTotalCitations();  
}
```

Research data is stored in `ResearchProfile`.

Research Classes

`ResearchProfile` stores research information.

```
private int hIndex;  
private List<ResearchPaper> papers;  
private List<ResearchProject> projects;
```

`ResearchPaper` stores article data.

```
private String title;  
private List<Researcher> authors;  
private String journal;  
private int pages;
```

```
private LocalDate publicationDate;
private String doi;
private int citations;
```

`ResearchProject` stores topic, participants, and published papers.

Only researchers can join research projects.

Research Rules

Important research rules:

1. Professors are always researchers.
2. Students and teachers can also be researchers.
3. A fourth-year student can have a research supervisor.
4. Supervisor must have h-index at least 3.
5. Only researchers can join research projects.

H-index check:

```
if (supervisor.getResearchProfile().getHIndex() < 3) {
    throw new LowHIndexException("Supervisor must have h-index at least 3");
}
```

Project participant check:

```
if (!(user instanceof Researcher)) {
    throw new NotResearcherException("Only researchers can join research projects");
}
```

Design Patterns

We used several design patterns.

Pattern	Class	Purpose
Singleton	<code>DataStore</code>	One shared storage
Factory Method	<code>UserFactory</code>	Creates users
Strategy	Comparators	Sorts research papers
Observer	<code>NewsService</code>	Publishes news

Pattern	Class	Purpose
Facade	<code>UniversitySystem</code>	Simplifies system usage

Example of Strategy pattern:

```
new PaperCitationsComparator();
new PaperDateComparator();
new PaperPagesComparator();
```

Storage and Serialization

`DataStore` stores the main data of the system:

- users;
- courses;
- research papers;
- research projects;
- registration requests;
- news;
- logs.

`DataStore` is implemented as Singleton, so the system has one shared storage.

The system also supports serialization. Main model classes implement `Serializable`, so data can be saved and loaded.

Services

Service classes contain business logic.

Main services:

- `AuthService` — login and password checking;
- `CourseService` — course registration and approval;
- `ResearchService` — research operations;
- `ReportService` — academic reports;
- `LogService` — user action logs;
- `NewsService` — news publishing.

This separation makes the project cleaner: models store data, services perform operations.

Demonstration

The `Main` class demonstrates the main functionality of the project.

The demo includes:

- creating users;
- authentication;
- creating courses;
- assigning teachers;
- registration request and approval;
- putting marks;
- transcript generation;
- adding research papers;
- sorting papers;
- creating research projects;
- testing custom exceptions;
- saving/loading data;
- printing logs.

The console version also supports role-based actions after login.

Insert: screenshot of terminal output.

Documentation

HTML documentation was generated using Javadoc.

It contains information about:

- classes;
- constructors;
- methods;
- fields.

Documentation location:

docs/index.html

This documentation is included because the final project requires HTML documentation for several classes.

Insert: Javadoc screenshot.

Project Management

The project was completed step by step.

Process:

Requirements → Diagrams → Draft classes → Full implementation → Testing → Documentation → Report

First, we analyzed project requirements. Then we created Use Case and UML Class diagrams. After that, we implemented Java classes and tested the main functionality through console demonstration.

The work was divided into logical parts: diagrams, models, services, research functionality, documentation, and report.

Problems

During development, we faced several problems.

1. It was difficult to choose the correct design for `Researcher`. The final solution was to make it an interface.
2. UML diagram and Java code had to stay consistent, so some classes were added after analysis.
3. Serialization required all connected model classes to implement `Serializable`.
4. It was important to separate model classes from service classes.
5. Research functionality required careful validation, especially h-index and project participation rules.

These problems helped improve the final architecture of the system.

Conclusion

The Research-Oriented University System implements the main requirements of the final OOP and Design project.

The system includes academic, administrative, and research functionality.

The project demonstrates:

- encapsulation;
- inheritance;
- abstraction;
- polymorphism;
- interfaces;
- collections;

- custom exceptions;
- comparators;
- serialization;
- design patterns.

The final implementation is consistent with the Use Case and UML Class diagrams and can be demonstrated through the console.